

This assignment is concerned with the mathematical description and definition of inverse kinematics of serial link robotic arms.

The following exercises are solved with Python in Jupyter notebooks and utilize standard scientific and robotics libraries (e.g., NumPy, SpatialMath, and the Robotics Toolbox for Python). Some tasks involve the use of ROS 2 (Jazzy) on a Linux (Ubuntu) system which is also available in the Retina Pool.

Learning Objectives: Inverse Kinematics Lab

After completing the *Inverse Kinematics* lab, students will have a thorough understanding of the mathematical principles, solution methods, and computational techniques required to solve the inverse kinematics (IK) problem for serial robotic manipulators. They will be able to determine the joint configurations needed to achieve a desired end-effector position and orientation in task space, building on the knowledge acquired in forward kinematics.

Students will understand the fundamental difference between forward and inverse kinematics, and the significance of the inverse kinematics problem in robotic motion planning and control. They will learn how to formulate the IK problem as a mapping from a target pose ${}^b\mathbf{H}_e = \{\mathbf{R}, \mathbf{t}\}$ to the joint vector $\mathbf{q} = [q_1, q_2, \dots, q_n]^T$, and will recognize the challenges associated with multiple solutions, non-uniqueness, and the existence of solutions depending on the robots workspace.

A key objective of the lab is to derive and apply *analytical* inverse kinematic solutions for simple robot configurations (e.g., planar manipulators or robots with spherical wrists). Students will learn how to use geometric approaches to determine joint angles from known end-effector positions and gain insight into how manipulator geometry affects the number and nature of IK solutions.

In addition to analytical approaches, students will develop skills in solving the inverse kinematics problem using *numerical optimization* methods. They will learn to formulate the IK problem as a nonlinear least-squares optimization task and to minimize pose errors that include both position and orientation components. They will understand how optimization techniques such as the Levenberg-Marquardt algorithm are used to iteratively refine joint configurations and how to handle cases where closed-form solutions are not available.

The lab also introduces practical tools from the Python robotics ecosystem to implement IK solvers. Students will learn how to configure solver parameters and weighting of pose errors, and compute feasible solutions given a target pose. They will be able to analyze and interpret the results, validate them through forward kinematics, and visualize robot motion and configurations in Python.

By the end of the lab, students will be able to:

- Explain the inverse kinematics problem and its significance in robot motion planning and control.

- Formulate and solve analytical IK problems for planar and spherical wrist manipulators.
- Recognize and handle multiple, redundant, or non-existent IK solutions.
- Implement numerical solutions to IK problems using nonlinear least-squares optimization methods.
- Use Python-based robotics tools to configure and solve IK problems, including pose error minimization.
- Validate IK solutions through forward kinematics and visualize results in Python.

These skills are essential for advanced topics in robotics such as trajectory planning, motion control, redundancy resolution, and optimization-based robot behavior, and they provide the foundation for autonomous and precise robotic manipulation.

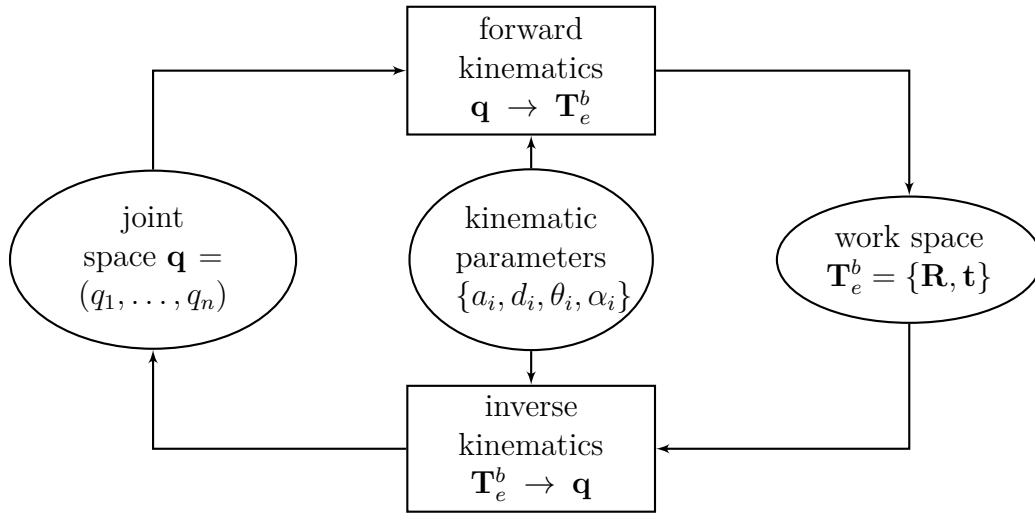


Figure 1: Forward and Inverse Kinematics

Inverse Kinematics

In many robotic applications, the desired motion of a manipulator is naturally specified in terms of the position and orientation of its end-effector rather than by individual joint angles. Consider, for example, a pick-and-place task in which a robot has to grasp an object at a known position and orientation. The desired grasp determines a target pose for the end-effector, but the robot controller ultimately requires corresponding values for the individual joints. The problem is therefore to determine which joint configuration places the end-effector at the desired pose.

This problem is referred to as *inverse kinematics*. While forward kinematics determines the end-effector pose from a given joint configuration, inverse kinematics considers the opposite direction: a desired end-effector pose is specified and suitable joint variables have to be determined. Inverse kinematics is therefore an essential component of robot motion planning, trajectory generation, manipulation, and control.

Problem Definition

Consider a serial manipulator with n joint variables collected in the joint configuration vector

$$\mathbf{q} = [q_1 \quad q_2 \quad \cdots \quad q_n]^T. \quad (1)$$

The forward kinematics defines a mapping from the joint space to the pose of the end-effector,

$$\mathbf{q} \mapsto {}^b\mathbf{T}_e(\mathbf{q}), \quad (2)$$

where

$${}^b\mathbf{T}_e(\mathbf{q}) = \begin{bmatrix} {}^b\mathbf{R}_e(\mathbf{q}) & {}^b\mathbf{t}_e(\mathbf{q}) \\ \mathbf{0}^T & 1 \end{bmatrix} \quad (3)$$

denotes the homogeneous transformation describing the position and orientation of the end-effector frame e with respect to the robot base frame b .

In the inverse kinematic problem, a desired end-effector pose ${}^b\mathbf{T}_{e,d}$ is given and the objective is to determine a joint configuration $\mathbf{q}^*$ such that

$${}^b\mathbf{T}_e(\mathbf{q}^*) = {}^b\mathbf{T}_{e,d}. \quad (4)$$

Equivalently, inverse kinematics can be regarded as solving the nonlinear system of equations

$$\mathbf{f}(\mathbf{q}) = \mathbf{x}_d, \quad (5)$$

where $\mathbf{f}$ represents the forward kinematic mapping and $\mathbf{x}_d$ represents the desired end-effector pose in a suitable position and orientation representation.

In contrast to forward kinematics, the inverse kinematic mapping cannot in general be expressed as a unique function

$$\mathbf{q} = \mathbf{f}^{-1}(\mathbf{x}_d), \quad (6)$$

because a unique inverse of the forward kinematic mapping does not necessarily exist. Depending on the manipulator geometry and the desired pose, the inverse kinematic problem may have no solution, one solution, several solutions, or even infinitely many solutions.

Number of Inverse Kinematic Solutions

A desired end-effector pose has *no solution* if it lies outside the reachable workspace of the manipulator or if the required orientation cannot be achieved at the desired position. Joint limits and other kinematic constraints can further reduce the set of feasible solutions.

A pose may have a *single solution* if exactly one joint configuration satisfies the desired end-effector pose within the admissible joint range. However, many serial manipulators can reach the same end-effector pose using different joint configurations. In this case, the inverse kinematic problem has *multiple solutions*. A common example is an anthropomorphic robot arm that can reach the same position using different shoulder or elbow configurations, such as an “elbow-up” and an “elbow-down” configuration.

If a manipulator possesses more degrees of freedom than are required to describe the specified task, it is referred to as a *kinematically redundant manipulator*. For example, a general pose in three-dimensional space requires six independent coordinates, consisting of three coordinates for position and three for orientation. A manipulator with more than six independent joints is therefore redundant with respect to a general six-dimensional end-effector pose.

For a redundant manipulator, a desired end-effector pose may correspond to *infinitely many joint configurations*. The additional degrees of freedom can then be exploited to

satisfy secondary objectives while maintaining the desired end-effector pose. Typical secondary objectives include avoiding joint limits, avoiding obstacles, increasing the distance from singular configurations, or minimizing joint motion.

Consequently, solving inverse kinematics does not necessarily mean finding *the* joint configuration. In many cases, the problem is to find one suitable configuration from a set of possible solutions.

Solution Methods

Inverse kinematic methods can broadly be divided into analytical, geometric, and numerical approaches. The appropriate method depends strongly on the kinematic structure of the manipulator.

Analytical Methods Analytical methods derive explicit equations for the joint variables from the forward kinematic equations. The elements of the desired homogeneous transformation matrix are compared with the corresponding elements of the forward kinematic transformation, and the resulting nonlinear equations are solved algebraically for the individual joint variables.

If a closed-form solution exists, analytical methods can determine the joint configurations directly and are generally computationally efficient. They can also provide all discrete solutions of the inverse kinematic problem. However, the derivation is highly dependent on the particular geometry of the robot, and a convenient closed-form solution does not exist for every manipulator.

Geometric Methods Geometric methods exploit the particular geometric structure of a manipulator. Instead of directly solving all elements of the homogeneous transformation matrix, the robot geometry is decomposed into simpler geometric problems that can be solved using trigonometric relationships.

For example, the position of a wrist center may first be determined from the desired end-effector pose. Joint angles controlling the arm can subsequently be obtained from triangles formed by the robot links. The remaining joints are then used to obtain the desired end-effector orientation. Such a decomposition is particularly useful for manipulators with suitable geometric structures, such as a spherical wrist.

Geometric approaches usually provide intuitive insight into the different possible robot configurations and make the existence of multiple solutions, such as elbow-up and elbow-down configurations, particularly apparent.

Numerical Methods Numerical inverse kinematics does not require an explicit closed-form inverse of the forward kinematic equations. Instead, an initial joint configuration $\mathbf{q}_0$ is iteratively modified until the resulting end-effector pose approaches the desired pose.

A numerical inverse kinematic problem can, for example, be formulated as the minimization of a pose error

$$\mathbf{q}^* = \arg \min_{\mathbf{q}} \|\mathbf{e}(\mathbf{q})\|^2, \quad (7)$$

where $\mathbf{e}(\mathbf{q})$ measures the difference between the desired and current end-effector pose. Alternatively, differential methods use the manipulator Jacobian to iteratively determine changes of the joint configuration,

$$\Delta \mathbf{q} = \mathbf{J}^\dagger(\mathbf{q}) \mathbf{e}, \quad (8)$$

where $\mathbf{J}^\dagger$ denotes the inverse or pseudoinverse of the Jacobian.

Numerical methods are applicable to a wide range of manipulator structures and can readily incorporate joint limits and additional constraints. Their result, however, generally depends on the initial configuration, convergence parameters, and the properties of the kinematic problem. In particular, an iterative method usually determines one locally accessible solution rather than automatically identifying all possible inverse kinematic solutions.

For redundant manipulators, the Jacobian pseudoinverse also provides a means of exploiting the additional degrees of freedom. Joint motion in the null space of the Jacobian can be used to optimize secondary objectives without changing the end-effector motion. Numerical methods are therefore especially important for inverse kinematics of redundant and constrained robotic manipulators.

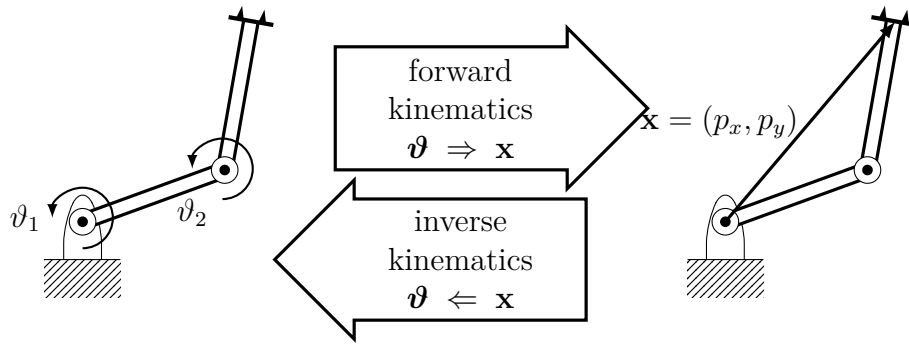


Figure 2: Forward and Inverse Kinematics Two Link Planar Arm

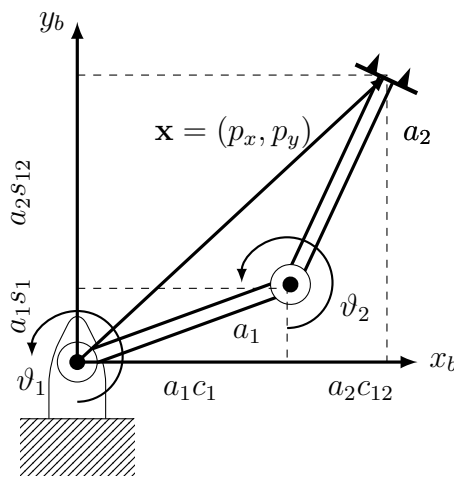


Figure 3: Forward Kinematics Two Link Planar Arm

Inverse Kinematics Two Link Planar Arm

Figure 2 illustrates the inverse kinematic problem for the two link planar arm. Given the end effector position $\mathbf{x} = (p_x, p_y)$ in the x - y -plane, determine the joint configuration ϑ_1, ϑ_2 to acquire that pose.

Figure 3 illustrates the relationship between joint variables and end effector pose imposed by the forward kinematics.

$$\mathbf{x} = \begin{bmatrix} p_x \\ p_y \end{bmatrix} = \begin{bmatrix} a_1 c_1 + a_2 c_{12} \\ a_1 s_1 + a_2 s_{12} \end{bmatrix} \quad (1)$$

To solve these two equations w.r.t. the two unknowns ϑ_1, ϑ_2 one squares and sums both equations in order to eliminate ϑ_1

$$\begin{aligned} p_x^2 + p_y^2 &= (a_1 c_1 + a_2 c_{12})^2 + (a_1 s_1 + a_2 s_{12})^2 \\ &= (a_1 c_1 + a_2 (c_1 c_2 - s_1 s_2))^2 + (a_1 s_1 + a_2 (s_1 c_2 + c_1 s_2))^2 \\ &= a_1^2 + a_2^2 + 2a_1 a_2 c_2 \end{aligned}$$

The resulting equation is rearranged to isolate ϑ_2 on the l.h.s.

$$\begin{aligned} c_2 &= \frac{p_x^2 + p_y^2 - a_1^2 - a_2^2}{2a_1a_2} \\ s_2 &= \pm\sqrt{1 - c_2^2} \\ \vartheta_2 &= \text{atan2}(s_2, c_2) \end{aligned} \quad (2)$$

The two alternative solutions correspond to the elbow up and elbow down posture. After solving for ϑ_2 we seek the remaining unknown ϑ_1 by substituting ϑ_2 into equations(1) to obtain

$$\begin{aligned} s_1 &= \frac{(a_1 + a_2c_2)p_y - a_2s_2p_x}{p_x^2 + p_y^2} \\ c_1 &= \frac{(a_1 + a_2c_2)p_x + a_2s_2p_y}{p_x^2 + p_y^2} \\ \vartheta_1 &= \text{atan2}(s_1, c_1) \end{aligned} \quad (3)$$

Python symbolic calculations (e.g., with SymPy) are helpful to find solutions of algebraic equations such as (1) solving for the unknowns ϑ_1, ϑ_2 given the known parameters p_x, p_y, a_1, a_2 .

```
import sympy as sp

a1, a2, theta1, theta2, px, py = sp.symbols("a1 a2 theta1 theta2 px py", real=True)

# solve direct kinematic equations for theta1, theta2
sol = sp.solve(
    [
        sp.Eq(px, a1*sp.cos(theta1) + a2*sp.cos(theta1 + theta2)),
        sp.Eq(py, a1*sp.sin(theta1) + a2*sp.sin(theta1 + theta2)),
    ],
    [theta1, theta2],
    dict=True
)
sol
```

To inspect and simplify the solution (note: SymPy returns multiple solutions, corresponding to elbow-up and elbow-down posture):

```
theta2_solutions = [s[theta2] for s in sol]
theta2_solutions

[sp.simplify(sp.cos(expr)) for expr in theta2_solutions]
```

which coincides with the analytical solution in Eq. (3).

The solution for ϑ_1 becomes

```
theta1_solutions = [s[theta1] for s in sol]
[sp.simplify(expr) for expr in theta1_solutions]
```

or in mathematical notation.

$$\vartheta_1 = 2 \operatorname{atan} \frac{2 a_1 p_y \pm \sqrt{-a_1^4 + 2 a_1^2 a_2^2 + 2 a_1^2 p_x^2 + 2 a_1^2 p_y^2 - a_2^4 + 2 a_2^2 p_x^2 + 2 a_2^2 p_y^2 - p_x^4 - 2 p_x^2 p_y^2 - p_y^4}}{a_1^2 + 2 a_1 p_x - a_2^2 + p_x^2 + p_y^2}$$

Notice, that you can generate the LaTeX code of a symbolic expression such as `sp.simplify(theta1_solutions[0])` with

```
sp.latex(sp.simplify(theta1_solutions[0]))
```

The solution looks more complicated than Eq. (4) as it is solely expressed in terms of p_x, p_y, a_1, a_2 , whereas the analytical solution in 4 refers to the previous solution ϑ_2 .

Numerical Solution to Inverse Kinematic Problem

Numerical solutions are universal as they rely on numerical algorithms, and provide solutions even if no closed-form solution is available. For a given pose there might be multiple solutions, e.g. elbow-up and elbow-down posture, or no solution at all, e.g. if the end effector pose is outside the manipulators workspace.

For a numerical solution the inverse kinematics problem is formulated as an optimization problem. In fact the objective is to minimize the error between the target transform $\mathbf{H}_t$ for the end effector and the forward kinematics of a joint configuration $\mathbf{H}_e(\mathbf{q})$. For that purpose we decompose the pose error into a position and a rotation part. The position error is merely the Euclidean distance between the origin of both frames, the length of the vector $\mathbf{p}_t - \mathbf{p}_e$.

$$\mathbf{e}_p(\mathbf{q}) = [e_x \ e_y \ e_z]^T = [p_{xt} - p_{xe} \ p_{yt} - p_{ye} \ p_{zt} - p_{ze}]^T \quad (4)$$

with

$$[p_{xt} \ p_{yt} \ p_{zt}]^T = [\mathbf{H}_t(1,4) \ \mathbf{H}_t(2,4) \ \mathbf{H}_t(3,4)]^T$$

and

$$[p_{xe} \ p_{ye} \ p_{ze}]^T = [\mathbf{H}_e(\mathbf{q})(1,4) \ \mathbf{H}_e(\mathbf{q})(2,4) \ \mathbf{H}_e(\mathbf{q})(3,4)]^T$$

For the rotation part the relative orientation matrix $R_d = R_t R_e(\mathbf{q})'$ is mapped to an axis angle representation $[\alpha \ w_x \ w_y \ w_z]^T$. The orientation error is given by

$$\mathbf{e}_w(\mathbf{q}) = [e_{wx} \ e_{wy} \ e_{wz}]^T = [\alpha w_x \ \alpha w_y \ \alpha w_z]^T \quad (5)$$

The overall error is a six-dimensional vector $\mathbf{e}(\mathbf{q}) = [e_x \ e_y \ e_z \ e_{wx} \ e_{wy} \ e_{wz}]^T$. In Python, a common representation of this pose error between two transforms is a 6D “twist”-like vector (translation and rotation), which can be computed from the relative transform. For example, with SpatialMath one can compute a 6D pose error via a helper such as `tr2delta(T_current, T_target)`:

```
import numpy as np
from spatialmath.base import tr2delta

# Example: pose error between current and target transforms (4x4)
# e = tr2delta(T_current, T_target) # returns a 6D vector [ex, ey, ez, ewx, ewy, ewz]
```

The objective is to minimize the error norm in the least squares sense.

$$\min_{\mathbf{q}} \frac{1}{2} \|\mathbf{e}(\mathbf{q})\|^2 = \min_{\mathbf{q}} \frac{1}{2} (e_x^2 + e_y^2 + e_z^2 + e_{wx}^2 + e_{wy}^2 + e_{wz}^2) \quad (6)$$

A more general error is obtained by weighting the individual errors

$$\min_{\mathbf{q}} \frac{1}{2} \mathbf{e}_w(\mathbf{q}) = \min_{\mathbf{q}} \frac{1}{2} \mathbf{e}' \mathbf{W} \mathbf{e} \quad (7)$$

in which $\mathbf{W}$ is a positive definite matrix, often diagonal. In fact if a feasible solution $\mathbf{q}^*$ of the inverse kinematics problem exists, namely the target pose is within the robot workspace then the pose error becomes zero

$$\mathbf{e}(\mathbf{q}^*) = \mathbf{0} \quad (8)$$

The problem (7) constitutes a non-linear least squares problem for which efficient optimization algorithms exist. The Jacobian is the matrix of first order derivatives of a vector valued function. In our case we are interested in partial derivatives of the error vector w.r.t. joint angles $J_{ij} = \frac{\partial e_i}{\partial q_j}$ which constitute the elements of the Jacobian $\mathbf{J}$.

The current solution $\mathbf{q}$ is improved with a Levenberg-Marquardt (damped least squares) step $\mathbf{q}' = \mathbf{q} + \Delta\mathbf{q}$ with $\Delta\mathbf{q}$ obtained from the algebraic solution of

$$(\mathbf{J}^T \mathbf{J} + \lambda \mathbf{I}) \Delta\mathbf{q} = \mathbf{J}^T \mathbf{e} \quad (9)$$

Inverse Kinematics of the myCobot 280 in Python/Jupyter

In this lab, the inverse kinematics of the six-degree-of-freedom *myCobot 280* manipulator is investigated numerically using Python in a Jupyter notebook. The objective is to determine a joint configuration $\mathbf{q}$ for a specified desired position and orientation of the end-effector. In addition to computing an inverse kinematic solution, the accuracy of the resulting configuration is evaluated using forward kinematics, and different numerical solution methods are compared.

The kinematic model of the myCobot 280 is imported from its URDF description using the Robotics Toolbox for Python. The imported model contains the kinematic structure of the robot, including its links and joints, and provides the forward kinematics required by the numerical inverse kinematics algorithms. Students first inspect the imported robot model and identify the relevant joint and end-effector frames.

A desired end-effector pose is subsequently specified as a homogeneous transformation

$$\mathbf{T}^* = \begin{bmatrix} \mathbf{R}^* & \mathbf{d}^* \\ \mathbf{0}^T & 1 \end{bmatrix}, \quad (10)$$

where $\mathbf{d}^*$ denotes the desired end-effector position and $\mathbf{R}^*$ its desired orientation. The target transformation is constructed in Python using the **SE3** representation provided by SpatialMath.

The first inverse kinematic solution is obtained using the Levenberg–Marquardt inverse kinematics solver **IK_LM** provided by the Robotics Toolbox. Starting from an initial joint configuration $\mathbf{q}_0$, the solver iteratively searches for a configuration $\mathbf{q}_{\text{IK}}$ that minimizes the difference between the desired and actual end-effector pose. A mask

$$\mathbf{w} = [w_x \ w_y \ w_z \ w_{\text{roll}} \ w_{\text{pitch}} \ w_{\text{yaw}}] \quad (11)$$

specifies which translational and rotational components of the pose are considered by the inverse kinematics solver. For a complete six-dimensional pose, all three position and all three orientation components are included.

After determining $\mathbf{q}_{\text{IK}}$, students verify the solution by applying forward kinematics,

$$\mathbf{T}_{\text{IK}} = \mathbf{T}(\mathbf{q}_{\text{IK}}), \quad (12)$$

and comparing the resulting end-effector pose with the desired pose $\mathbf{T}^*$. The difference is represented by a six-dimensional pose error vector

$$\mathbf{e} = [e_x \ e_y \ e_z \ e_{\text{roll}} \ e_{\text{pitch}} \ e_{\text{yaw}}]^T. \quad (13)$$

The norm $\|\mathbf{e}\|$ provides a quantitative measure for evaluating how closely the numerical IK solution reproduces the specified target pose. The resulting robot configuration is additionally visualized using the plotting functionality of the Robotics Toolbox.

In the second part of the assignment, inverse kinematics is formulated as a general non-linear optimization problem. Instead of relying exclusively on a dedicated robotics IK solver, students define an objective function based on the end-effector pose error. The scalar objective

$$J(\mathbf{q}) = \mathbf{e}^T(\mathbf{q})\mathbf{e}(\mathbf{q}) = \sum_i e_i^2(\mathbf{q}) \quad (14)$$

is minimized using the BFGS algorithm provided by `scipy.optimize.minimize`. In addition, the pose error vector is directly used as the residual of a nonlinear least-squares problem,

$$\mathbf{q}^* = \arg \min_{\mathbf{q}} \|\mathbf{e}(\mathbf{q})\|^2, \quad (15)$$

which is solved using the Levenberg–Marquardt method provided by `scipy.optimize.least_squares`.

The assignment therefore produces three numerical inverse kinematics solutions: a solution obtained with the Robotics Toolbox IK_LM solver, a solution obtained by BFGS optimization, and a solution obtained by Levenberg–Marquardt nonlinear least squares. For each solution, students compute the corresponding end-effector pose using forward kinematics and determine the resulting pose error.

Finally, the three approaches are compared with respect to the achieved pose error. Students should analyze whether the methods converge to the same or different joint configurations and compare how accurately the resulting configurations reproduce the desired end-effector pose. This comparison also illustrates an important characteristic of numerical inverse kinematics: the computed solution may depend on the selected algorithm, the initial joint configuration, and the formulation of the pose error.

Python Inverse Kinematic Solvers

Robotics Toolbox for Python provides inverse kinematics (IK) solvers to calculate joint configurations for a desired end effector pose based on a specified robot model (e.g., the myCobot 280 model used in this lab). The following example generates a numerical IK solution for a target pose `targetpose`. In this case, `targetpose` originates from a random configuration and forward kinematics.

```
import numpy as np
import roboticstoolbox as rtb

# robot: provided myCobot 280 model in the template/notebook
# robot = ...

weights = np.ones(6)          # pose error weighting (translation + rotation)
q0 = robot.qz                 # initial guess (e.g., "zero" pose)
q_rand = robot.random_q()     # random joint vector within limits

targetpose = robot.fkine(q_rand) # forward kinematics gives SE3 target pose

# Numerical IK (Levenberg-Marquardt style solver)
sol = robot.ikine_LM(targetpose, q0=q0, mask=weights)

q_sol = sol.q                 # joint vector solution
# robot.plot(q_sol)           # visualize configuration (if enabled)
```

To evaluate the pose error in task space, map the joint configuration `q_sol` into the task space transform with forward kinematics and determine its error w.r.t. the target transform `targetpose`:

```
from spatialmath.base import tr2delta

T_sol = robot.fkine(q_sol)    # SE3
T_tgt = targetpose            # SE3

# Convert to 4x4 arrays for tr2delta
poseerror = tr2delta(T_sol.A, T_tgt.A)
```

```
# joint space error (if the original random configuration is known)
jointerror = q_rand - q_sol
```

If the original configuration `q_rand` is known, the comparison can also be performed in the joint space between `q_rand` and `q_sol`. However, `q_rand` is usually unknown in real applications.

Please complete tasks 1 to 9 in the template.

Nonlinear Least Squares Problems

You might skip this section if you are already familiar with local optimization and in particular non-linear least squares problems from the courses 'Scientific Programming' or 'Datenbasierte Modellierung und Optimierung'.

In least-squares problems, the objective function f has a particular structure that is exploited by the optimization algorithm

$$f(x) = \frac{1}{2} \sum_{j=1}^m r_j(x)^2 \quad (16)$$

in which $\mathbf{r}(x) = (r_1(x), \dots, r_m(x))^T$ denotes a vector of residuals. Non-linear least squares problems are particular relevant for regression in which a parametrized model is identified from observed data. In this case $r_j(x)$ measures the disagreement between the prediction of the model and the observed output over a set of training data $\{(x_i, y_i)\}$. By minimizing Eq. (16), the parameters of the model are selected such that it matches the data in a least squares sense. Numerical optimization schemes such as Levenberg-Marquardt exploit the particular structure of f and its derivatives.

The gradient of f is given by

$$\nabla f(x) = \sum_{j=1}^m r_j(x) \nabla r_j(x) = J(x)^T r(x) \quad (17)$$

in which the Jacobian $J(x) = [\frac{\partial r_j(x)}{\partial x_i}]_{i=1,2,\dots,n}^{j=1,2,\dots,m}$ denotes the matrix of first order partial derivatives.

In least-squares problems the Hessian $\nabla^2 f(x)$ matrix can be approximated from the knowledge of the Jacobian.

$$\nabla^2 f(x) = \sum_{j=1}^m \nabla r_j(x) \nabla r_j(x)^T + \sum_{j=1}^m r_j(x) \nabla^2 r_j(x)^T \quad (18)$$

$$= J(x)^T J(x) + \sum_{j=1}^m r_j(x) \nabla^2 r_j(x)^T \quad (19)$$

The first term $J(x)^T J(x)$ often dominates the second term in particular if the residuals $r_j(x)$ become small. Algorithms such as Levenberg-Marquardt exploit this property of the Hessian for non-linear least squares problems.

Levenberg-Marquardt Algorithm

The Levenberg-Marquardt algorithm detailed in 1 constitutes a trust-region method. The Levenberg-Marquardt so called dog leg step $\mathbf{p}^{LM}$ is an interpolation between the gradient descent direction

$$\mathbf{p}_k^{GD} = -\mathbf{J}_k^T \mathbf{r}_k$$

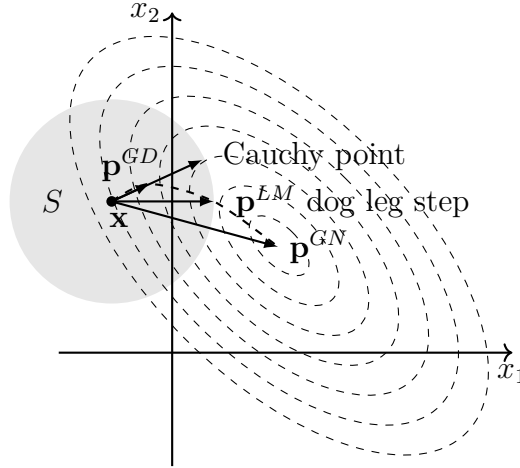


Figure 4: Levenberg Marquardt dog leg step

and the Gauss-Newton direction

$$\mathbf{p}_k^{GN} = -(\mathbf{J}_k^T \mathbf{J}_k)^{-1} \mathbf{J}_k^T \mathbf{r}_k$$

as shown in figure 4. As $\lambda \rightarrow 0$ the Levenberg-Marquardt step $\mathbf{p}^{LM}$ favors the Gauss-Newton direction $\mathbf{p}^{GN}$. The dog leg is shown as the dashed line in figure 4 that results from the variation of $\lambda \rightarrow 0$. The gray circle S_k denotes the trust-region and which restricts $p_k^{LM} \in S_k$.

The Levenberg-Marquardt algorithm solves non-linear least squares problems that arise in least squares regression. The Levenberg-Marquardt method can be interpreted as a trust region approach. Assuming a spherical trust region S_k of radius Δ_k each Levenberg-Marquardt step solves the quadratic problem

$$\min_p \frac{1}{2} \|\mathbf{J}_k p + \mathbf{r}_k\|^2 \text{ with } p \in S_k \quad (20)$$

This is equivalent of choosing a model function:

$$m_k(p) = \frac{1}{2} \|\mathbf{r}_k\|^2 + p^T \mathbf{J}_k^T \mathbf{r}_k + \frac{1}{2} p^T \mathbf{J}_k^T \mathbf{J}_k p \quad (21)$$

The optimal Gauss-Newton step of Eq. (21) is given by

$$\mathbf{J}_k^T \mathbf{J}_k p_k^{GN} = -\mathbf{J}_k^T \mathbf{r}_k \quad (22)$$

In case $p_k^{GN} \in S_k$ ($\|p_k^{GN}\| < \Delta$) it coincides with the solution of Eq. (21). Otherwise, there is a $\lambda > 0$ such that the solution $p = p^{LM}$ of Eq. (21) satisfies $\|p\| = \Delta$ and

$$(\mathbf{J}_k^T \mathbf{J}_k + \lambda \mathbf{I})p = -\mathbf{J}_k^T \mathbf{r}_k. \quad (23)$$

Algorithm 1 Levenberg-Marquardt

```

1:  $k \leftarrow 0$ ;
2:  $\lambda \leftarrow \max \text{diag}(J^T J)$ ;
3:  $x \leftarrow x_0$ ;
4: while STOP-CRIT and  $k < k_{max}$  do
5:   Find  $p$  such that  $(J^T J + \lambda \text{diag}(J^T J))p = J^T r$ 
6:    $x' \leftarrow x + p$ ;
7:   if  $f(x') \leq f(x)$  then
8:      $x \leftarrow x'$ ;
9:      $\lambda \leftarrow \frac{\lambda}{\nu}$ ;
10:  else
11:     $\lambda \leftarrow \nu \lambda$ ;
12:   $k \leftarrow k + 1$ ;
13: return  $x$ ;

```

Levenberg-Marquardt utilizes $\text{diag}(J_k^T J_k)$ rather than the identity matrix I .

$$(J_k^T J_k + \lambda \text{diag}(J_k^T J_k))p = -J_k^T r_k. \quad (24)$$

In practice λ is either determined by a Cholesky decomposition of $(J_k^T J_k + \lambda I)$ or by an adaptation rule that reduces λ if the new solution $x_{k+1} = x_k + p_k$ improves the objective function, or vice versa increases λ in case the new solution is inferior compared to the current solution x_k (see algorithm 1).

Python Nonlinear Least Squares Optimization

In Python, `scipy.optimize.least_squares` solves non-linear least-squares problems of the form in Eq. (16). You can provide optional lower and upper bounds `lb` and `ub` on the components of x .

Rather than to compute the sum of squares $f(x)$, `least_squares` requires the user-defined function to compute the vector-valued function of residuals $\mathbf{r}(x) = (r_1(x), \dots, r_m(x))^T$. `x = least_squares(fun, x0)` starts at the point `x0` and finds a minimum of the sum of squares of the functions described in `fun`. The function `fun` should return a vector of values and not the sum of squares of the values, in case of the Rosenbrock function $r_1(x_1, x_2) = 10(x_2 - x_1^2)$, $r_2(x_1, x_2) = 1 - x_1$. Common choices for the solver are `method='lm'` (Levenberg-Marquardt) or `method='trf'` (trust-region reflective). Notice that `'lm'` does not handle bounds, while `'trf'` supports bounds.

Numerical Solution of Inverse Kinematics

This assignment deals with the numerical solution of inverse kinematics with the help of Python numerical optimization. The solution of the inverse kinematics problem is a non-linear least squares problem of equation (16), in particular the cost function w.r.t.

joint vector $\mathbf{q}$ becomes

$$E(\mathbf{q}) = \frac{1}{2}(e_x^2 + e_y^2 + e_z^2 + e_{wx}^2 + e_{wy}^2 + e_{wz}^2) \quad (25)$$

namely to minimize the norm of the pose error in target space $\mathbf{e}$.

A typical implementation defines a residual function that returns the 6D pose error $\mathbf{e}(\mathbf{q})$ and then solves it with `least_squares`:

```
import numpy as np
from scipy.optimize import least_squares
from spatialmath.base import tr2delta

def residual(q, robot, T_target):
    T = robot.fkine(q)          # SE3
    return tr2delta(T.A, T_target.A) # 6D residual vector

# initial guess:
q0 = robot.qz

# solve:
res = least_squares(lambda q: residual(q, robot, targetpose), q0, method="lm")
q_star = res.x
```

Please complete tasks 10 to 14 in the template.

Competences

After successfully completing the Inverse Kinematics lab, students will have acquired the following competences:

- They understand the fundamental principles, mathematical formulation, and practical significance of the inverse kinematics (IK) problem in robotic motion planning and control.
- They can formulate the IK problem as a mapping from a desired end-effector pose to a joint configuration and explain the relationship between forward and inverse kinematics.
- They are able to derive and solve analytical inverse kinematics solutions for simple robot manipulators, such as planar arms and spherical wrist robots, and understand the impact of manipulator geometry on solution existence and multiplicity.
- They can analyze and handle multiple IK solutions, redundancy, and cases with no feasible solution, and evaluate workspace constraints.
- They are proficient in applying geometric and algebraic approaches to compute joint angles from known end-effector positions.

- They can formulate and solve the inverse kinematics problem as a nonlinear least-squares optimization problem and use numerical methods (e.g., Levenberg-Marquardt) to find approximate solutions when closed-form solutions do not exist.
- They are capable of implementing IK solvers using Python-based robotics tools, configuring solver parameters, defining weighting matrices, and minimizing pose errors in position and orientation.
- They can validate inverse kinematics solutions through forward kinematics, analyze the accuracy of solutions, and visualize robot motion and configurations in Python.
- They understand the role of the Jacobian matrix in IK optimization and can interpret its use in iterative solution refinement and convergence analysis.
- They have the skills to integrate IK solvers into broader robotic applications such as trajectory planning, redundancy resolution, and optimization-based control.